

Fase-2: PMN (Prototipo Mínimo Navegable)

Esta fase no consiste en crear la aplicación completa ni en lograr todas las funcionalidades posibles

Tiene un objetivo muy importante y estratégico, **lograr una maqueta** de la aplicación que permita **recorrer** el sistema, interactuar con él y comprender cómo funcionaría realmente

Es decir, una aplicación que ya **permita navegar entre pantallas**, realizar **algunas acciones importantes**, **visualizar** información (aunque se falsa), probar **decisiones**, recorrer **procesos** y percibir la **lógica general**. Aunque aún se tengan procesos incompletos, no existan validaciones, las funcionalidades se simuladas, los datos sean ficticios, existan partes sin terminar, etc.

Un PMN permite...

- detectar errores de diseño
- descubrir problemas de navegación
- identificar procesos absurdos
- detectar información faltante
- visualizar mejor el sistema
- mostrar la idea a otras personas
- recibir retroalimentación temprana
- comenzar a “sentir” la aplicación

A veces una buena idea de aplicación fracasa porque es incómoda de usar, confunde al usuario, tiene demasiados pasos, muestra información innecesaria, obligaba a navegar de manera poco natural

Un buen PMN no necesita ser visualmente perfecto, técnicamente complejo, completo o seguro. Pero debe lograr que alguien pueda usarlo y entender cómo funcionaría realmente la aplicación

Por eso un PMN debe tener **coherencia**, **navegación** clara, **lógica** comprensible, **simulación** de funcionamiento real, aunque internamente todavía esté en pañales y todo sea simulación

En esta etapa deben dejar de pensar solamente en diagramas, conceptos, descripciones, modelos y deben comenzar a pensar en **pantallas**, **botones**, **formularios**, **menús**, **navegación**, **experiencia del usuario**

Diferencia entre MPN y PMN

El foco del PMN fue conceptual y estuvo centrado en comprender el problema, modelar procesos, identificar actores, definir estados, identificar excepciones, entender el negocio. Ahora el foco cambia completamente pues deberán responder preguntas tales como ...

- ¿qué verá el usuario?
- ¿cómo navegará?
- ¿qué ocurrirá después de presionar un botón?
- ¿qué información debe aparecer?
- ¿qué decisiones tomará el usuario?
- ¿qué pantallas faltan?
- ¿cómo se vuelve atrás?
- ¿cómo se representa un estado?
- ¿qué acciones son realmente importantes?

Ahora el desafío ya no es “entender el problema” sino que “transformar eso en una aplicación navegable”

¿Cómo deben enfrentar esta fase?

No intenten crear demasiadas funcionalidades, desarrollar demasiados detalles, intentar terminar la aplicación pues eso los podría llevar a frustración, desorden, inconsistencias, pérdida de tiempo

La forma correcta es...

Elijan un recorrido principal- ¿Cuál es el recorrido más representativo e importante de la aplicación?

Por ejemplo, podría ser registrar una atención, crear un pedido, solicitar una herramienta, registrar un vehículo, aprobar una devolución

Luego construyan solo ese recorrido (no lograr la aplicación completa). **Deben lograr una navegación real** para que, al probar esa maqueta, se sienta que se avanza, se entra y sale de pantallas, ocurren acciones, aparecen resultados, que el sistema responde... aunque muchas o todas esas cosas estén simuladas

No es obligatorio que todo funcione completa o realmente. Pueden usar datos ficticios, botones simulados, mensajes temporales, listas simples, procesos representados solo parcialmente, etc.

Lo importante es que el recorrido general se entienda

Primero construyan lo visible pues, en esta etapa es más importante **lo que ve el usuario, lo que hace el usuario hace, lo que siente el usuario**.

No dediquen tiempo a codificar algoritmos complejos, bases de datos depuradas, optimizaciones técnicas

No intenten construir una **versión final** desde el inicio, Primero creen algo pequeño, pruébenlo, luego méjorenlo y agréguele cosas. De esa forma el PMN evolucionará rápidamente

La IA puede ayudarles a acelerar el desarrollo, en cosas tales como generar interfaces, crear formularios, conectar pantallas, proponer estructuras, corregir errores, mejorar la navegación, generar código

Pero delegarle todo a la IA tiene un riesgo, pues la aplicación podría verse impresionante, tener mucho código, tener excelente diseño visual, parecer profesional y, de todos modos, estar mal pensada, mal organizada, incoherente, incompleta, confusa

Por eso en esta fase es importante que interactúen intensamente con la IA para cuestionar decisiones, detectar errores, probar recorridos, pensar en el usuario, etc. No basta con que la IA genere algo, la aplicación debe tener sentido

Al revisar su PMN valoraré mucho:

- navegación clara
- coherencia general
- sensación de funcionamiento real
- buen recorrido principal
- decisiones bien pensadas
- mejora progresiva
- iniciativa
- capacidad de iteración
- capacidad de resolver problemas
- [Diálogos con la IA](#)

NO le daré tanto valor a valorar a tamaño de la aplicación, exceso de funcionalidades, complejidad innecesaria, interfaces exageradamente decoradas, cantidad absurda de código

En esta fase NO espero perfección, espero algo mucho más valioso... que comiencen a transformar una idea abstracta en una experiencia interactiva real (este paso es uno de los más difíciles e importantes en el desarrollo de software)

Objetivo final de esta fase

Al finalizar las dos semanas destinadas a esta fase deberían poder mostrar una aplicación donde alguien pueda recorrer pantallas, interactuar, ejecutar acciones principales, comprender la aplicación, percibir cómo funcionaría realmente una vez construida.

Muy importante, ustedes mismos deberían sentir que la aplicación **ya existe**, aunque todavía esté lejos de estar terminada

Ese es exactamente el propósito de un buen Prototipo Mínimo Navegable

Una perturbación inevitable

Aunque **esta fase se extiende formalmente por dos semanas**, la segunda corresponde a un receso académico, sin clases ni evaluaciones. Entonces, si no avanzan en la primera semana es probable que en la segunda lo hagan recién casi al vencer el plazo de entrega, lo cual sería un error fatal

El problema es que esta fase funciona de manera muy distinta a la anterior. En el MPN muchas veces era posible avanzar rápidamente generando modelos, diagramas o propuestas con ayuda intensa de IA

Pero en el PMN comenzará a ocurrir algo distinto, aparecerán errores inesperados, problemas de navegación, faltarán pantallas, aparecerán incoherencias, ciertas decisiones no tendrán sentido, algunas ideas sonarán bien, pero funcionarán mal, la IA generará soluciones incompletas, tendrán que probar, corregir y volver a intentar. Justamente por eso esta fase necesita tiempo real de exploración, prueba y ajuste

Un PMN rara vez aparece correctamente al primer intento y normalmente evoluciona a través de pruebas, errores, mejoras, simplificaciones, nuevas ideas. Por lo tanto, **mi recomendación es muy simple, no esperen intentar construir el PMN completo el último día.**

Lo más inteligente es comenzar temprano, aunque sea con algo extremadamente pequeño y simple. Por ejemplo, una primera pantalla, un menú, un formulario básico, una navegación mínima, un recorrido principal muy reducido

Porque una vez que algo comienza a funcionar aparecen ideas, aparecen mejoras, aparecen problemas reales, comienza a sentirse la aplicación

Y desde allí avanzar se vuelve mucho más fácil, en cambio, intentar construir todo al final normalmente conduce a frustración, aplicaciones incoherentes, navegación incompleta, errores difíciles de resolver, soluciones improvisadas, dependencia excesiva de respuestas rápidas generadas por IA

El objetivo de esta fase no es producir una aplicación gigantesca, es lograr un prototipo navegable que transmita claramente cómo funcionaría el sistema, cómo interactuaría el usuario, qué decisiones existirían, qué experiencia produciría la aplicación

Y eso requiere tiempo de prueba, exploración y evolución

Cómo relacionar correctamente el PMN con el MPN

Una buena forma de comenzar esta fase es volver al MPN y preguntarse:

- ¿Qué procesos realmente son centrales?
- ¿Qué actor realizará más acciones?
- ¿Qué decisiones importantes existen?
- ¿Qué estados deberían verse reflejados?
- ¿Qué excepciones podrían aparecer?
- ¿Qué información debe mostrarse?
- ¿Qué partes del proceso probablemente necesitan pantallas distintas?

Estrategia que les recomiendo

- Tomen el recorrido principal del MPN
- Imaginen cómo se recorrería eso en una aplicación real
- Definan pantallas, botones, formularios, menús, resultados, navegación
- Construyan primero una versión extremadamente simple
- Pruébenla y mejórenla

Algo importante

El PMN NO reemplaza al MPN, el PMN nace desde el MPN

El MPN representa comprensión, lógica, procesos, estructura del problema

El PMN representa experiencia, navegación, interacción, visualización concreta del sistema

Por eso muchas decisiones tomadas en el MPN comenzarán ahora a hacerse visibles y serán necesarias mejoras, ajustes, simplificaciones, nuevas ideas, errores que antes no eran evidentes

Eso no significa que MPN estuviese equivocado, significa que ahora estarán comenzando a transformar una idea conceptual en algo que ya puede recorrerse, probarse y sentirse como una aplicación real.

Herramientas que podrían ayudarles en esta Fase

No todos los grupos necesitarán las mismas herramientas. La idea es usar solamente las que les sean útil

la generativa

Puede ayudar muchísimo para generar código, crear interfaces, conectar pantallas, corregir errores, generar formularios, proponer mejoras, simplificar navegación

Herramientas visuales de diseño

Tales como **Figma**, **Canva**, **Excalidraw**, **Draw.io** especialmente para ordenar ideas, diseñar pantallas, visualizar navegación, pensar recorridos

Herramientas simples de desarrollo

Dependiendo de la tecnología que usen podrían ser HTML/CSS/JavaScript, Python, Streamlit, Flask, React, Firebase, Glide, Lovable, Bolt, Replit

Lo importante NO es usar la herramienta más profesional, lo importante es lograr avanzar rápidamente hacia algo navegable y comprensible.

Herramientas para registrar avances

Puedes ser muy útiles programas gratuitos para capturar pantallas, guardar versiones, registrar cambios importantes

REFLEXIONES NECESARIAS

Duda razonable

Es probable que varios grupos sientan ahora una inquietud muy razonable...

¿Cómo pasamos desde un MPN a un prototipo navegable, especialmente si parte importante de ese MPN fue construido con ayuda intensa de IA y no necesariamente comprendimos todo en profundidad?"

La respuesta es simple pero importante... **precisamente por eso existe esta nueva fase**

En el mundo real es muy frecuente que una idea, un modelo o incluso una propuesta técnicamente correcta “parezca clara” mientras permanece escrita en documentos, diagramas o descripciones. Pero cuando se intenta transformar esa idea en una aplicación usable, navegable, comienzan a aparecer preguntas, vacíos, contradicciones y decisiones que antes no eran evidentes

Si les ocurre eso, no significa que el MPN está mal, solo significa que comenzaron a recorrerlo de una forma distinta. Ante el foco estuvo principalmente en comprender el problema, identificar actores, definir procesos, detectar excepciones, modelar el negocio

Ahora el foco cambia hacia preguntas mucho más concretas ¿Qué verá primero el usuario? ¿Cómo navegará? ¿Qué ocurrirá después de presionar un botón? ¿Qué información debe aparecer? ¿Qué decisiones deberá tomar? ¿Qué partes del proceso realmente importan? ¿Qué ocurre si algo falla? ¿Qué pantallas faltan?

Entonces muchas partes de su MPN que parecían suficientemente claras deberán reinterpretarlas, ajustarlas, simplificarlas o incluso replantearlas para lograr una aplicación navegable y coherente

Eso es completamente normal

De hecho, una de las razones por las que existen prototipos navegables es precisamente para descubrir tempranamente problemas que todavía no eran visibles en los modelos conceptuales

Por eso esta fase **no debe enfrentarse** como “**ahora hay que programar todo**” sino como “**ahora debemos intentar convertir nuestro modelo en algo que pueda recorrerse, probarse y sentirse como una aplicación real**”

Inevitablemente aparecerán dudas, errores, inconsistencias, cambios, mejoras, nuevas ideas

Pero eso no representa un fracaso del trabajo anterior, representa exactamente el proceso normal mediante el cual evolucionan los sistemas reales

Inquietud razonable

Muchos de ustedes podrían estar haciéndose la siguiente pregunta **¿realmente es necesario, realmente es útil hacer todo esto sí, cuando esté ya esté titulado y ejerciendo en alguna empresa, probablemente nunca me van a solicitar que desarrolle una aplicación desde el modelado del proceso de negocio y, a lo mejor, lo más frecuente será tener que hacer modificaciones o mantención de software heredado (software Legacy)?**

Esa pregunta es completamente válida pues, en gran parte de los trabajos reales, especialmente al inicio de su carrera profesional, es mucho más frecuente que realizarán tareas tales como...

- modificar sistemas existentes
- corregir errores
- agregar funcionalidades pequeñas
- mantener software heredado
- adaptarse a tecnologías ya definidas
- trabajar sobre código escrito por otros

Pero precisamente por eso esta experiencia es importante

Porque incluso cuando alguien trabaja sobre sistemas heredados o sobre aplicaciones desarrolladas por otros, necesita comprender cosas como:

- qué problema intenta resolver el sistema
- qué lógica existe detrás de ciertas decisiones
- qué procesos representa
- qué actores participan
- qué información circula
- cómo navega el usuario
- qué estados existen
- qué partes son realmente importantes
- qué ocurrirá si algo cambia

Y esas cosas son mucho más difíciles de comprender cuando nunca se ha vivido el proceso de intentar construirlas

Muchas veces un profesional recién comprende realmente por qué ciertos sistemas son:

- incómodos
- caóticos
- difíciles de mantener
- incoherentes
- llenos de errores
- difíciles de modificar

Además, hay algo importante respecto del contexto tecnológico actual. [La IA está reduciendo cada vez más la dificultad técnica de generar código](#), entonces el valor profesional **comienza a desplazarse progresivamente hacia otras capacidades:**

- comprender problemas
- detectar incoherencias
- tomar decisiones
- diseñar experiencias
- cuestionar soluciones
- pensar procesos
- interpretar necesidades reales
- evaluar si algo tiene o no sentido

[Porque generar código ya no es la parte más difícil](#). Entonces, justamente por eso, estas fases no buscan lograr habilidades como desarrollador, sino que experimentar algo más importante, cómo una idea inicial comienza a transformarse en una aplicación real

Aunque en el futuro trabajen principalmente con sistemas existentes, esta experiencia les ayudará a:

- comprender mejor los sistemas
- detectar problemas
- proponer mejoras
- comunicarse con otros profesionales
- interpretar lógica de negocio
- entender decisiones de diseño

Porque detrás de todo sistema existe siempre una lógica, una intención, un conjunto de decisiones, una experiencia de usuario